Faculty of Engineering and Technology

Electrical and Computer Engineering Department

Information Retrieval with Applications of NLP

ENCS5342

Assignment 2

Prepared by:

Name: Ahmad Zuhd **ID**: 1222332

Name: Bara Mohsen ID: 1220829

Instructor: Dr. Ahmed I. A. Shawahna

Date : 25-4-2026

Section: 1

Task 1 — Let's Build a Language Model!

Step 1 — Tokenize the Text

```
##############################################
First - Tokenize the texts
perl tokenizer.pl -no-escape < ../English-Mix/UNCorpus.train > ../English-Mix/UNCorpus.train.tok
Tokenizer Version 1.1
Language: en
Number of threads: 1
perl tokenizer.pl -no-escape < ../English-Mix/UNCorpus.test > ../English-Mix/UNCorpus.test.tok
Tokenizer Version 1.1
Language: en
Number of threads: 1
perl tokenizer.pl -no-escape < ../English-Mix/Bible.test > ../English-Mix/Bible.test.tok
Tokenizer Version 1.1
Language: en
Number of threads: 1
##############################################
```

Q1.a: Effect Of Tokenization

The tokenizer (Moses tokenizer) applies the following changes to the raw text:

- Punctuation marks are separated from words with a space — e.g., "meeting," becomes "meeting ," and "follows:" becomes "follows :"
- Brackets and parentheses are split off — e.g., "(A/55/602)" becomes "(A / 55 / 602)"
- Slashes between numbers/codes are separated — e.g., "55/100" becomes "55 / 100"
- Known abbreviations and prefixes listed in nonbreaking_prefix.en (e.g., "Mr.", "Dr.", single uppercase letters) are NOT split, because they do not signal sentence endings
- The -no-escape flag prevents HTML entity encoding (e.g., & stays as & rather than &)

Example — Raw line:

Adopted at the 81st plenary meeting, on 4 December 2000, ... (A/55/602/Add.2 and Corr.1, para. 94),

Tokenized:

Adopted at the 81st plenary meeting , on 4 December 2000 , ... (A / 55 / 602 / Add.2 and Corr.1 , para . 94) ,

Q1.b — Token and Type Counts (Raw vs. Tokenized)

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                                    wsl

ahmad@DESKTOP-5SVTQJS:/mnt/c/Users/Ahmad Zuhd/Documents/4 year/NLP/ENCS5342-Assignment02-T1252/ENCS5342-Assignment02-T1252$ wc -w < English-Mix/UNCorpus.train
2600422
ahmad@DESKTOP-5SVTQJS:/mnt/c/Users/Ahmad Zuhd/Documents/4 year/NLP/ENCS5342-Assignment02-T1252/ENCS5342-Assignment02-T1252$ tr ' \t\n' '\n' <English-Mix/UNCorpus.train | sort
-u | wc -l
36466
ahmad@DESKTOP-5SVTQJS:/mnt/c/Users/Ahmad Zuhd/Documents/4 year/NLP/ENCS5342-Assignment02-T1252/ENCS5342-Assignment02-T1252$ wc -w < English-Mix/UNCorpus.train.tox
-bash: English-Mix/UNCorpus.train.tox: No such file or directory
ahmad@DESKTOP-5SVTQJS:/mnt/c/Users/Ahmad Zuhd/Documents/4 year/NLP/ENCS5342-Assignment02-T1252/ENCS5342-Assignment02-T1252$ wc -w < English-Mix/UNCorpus.train.tok
2933681
ahmad@DESKTOP-5SVTQJS:/mnt/c/Users/Ahmad Zuhd/Documents/4 year/NLP/ENCS5342-Assignment02-T1252/ENCS5342-Assignment02-T1252$ tr ' \t\n' '\n' <English-Mix/UNCorpus.train.tok | s
ort -u | wc -l
17860
ahmad@DESKTOP-5SVTQJS:/mnt/c/Users/Ahmad Zuhd/Documents/4 year/NLP/ENCS5342-Assignment02-T1252/ENCS5342-Assignment02-T1252$ wc -w < English-M
16693                                                                                  Open file in editor (ctrl + click)
ahmad@DESKTOP-5SVTQJS:/mnt/c/Users/Ahmad Zuhd/Documents/4 year/NLP/ENCS5342-Assignment02-T1252/ENCS5342-Assignment02-T1252$ tr ' \t\n' '\n' <English-Mix/UNCorpus.test | sort -
u | wc -l
2695
ahmad@DESKTOP-5SVTQJS:/mnt/c/Users/Ahmad Zuhd/Documents/4 year/NLP/ENCS5342-Assignment02-T1252/ENCS5342-Assignment02-T1252$ wc -w < English-Mix/UNCorpus.test.tok
19169
ahmad@DESKTOP-5SVTQJS:/mnt/c/Users/Ahmad Zuhd/Documents/4 year/NLP/ENCS5342-Assignment02-T1252/ENCS5342-Assignment02-T1252$ tr ' \t\n' '\n' <English-Mix/UNCorpus.test.tok | so
rt -u | wc -l
2155
ahmad@DESKTOP-5SVTQJS:/mnt/c/Users/Ahmad Zuhd/Documents/4 year/NLP/ENCS5342-Assignment02-T1252/ENCS5342-Assignment02-T1252$ wc -w < English-Mix/Bible.test
11995
ahmad@DESKTOP-5SVTQJS:/mnt/c/Users/Ahmad Zuhd/Documents/4 year/NLP/ENCS5342-Assignment02-T1252/ENCS5342-Assignment02-T1252$ tr ' \t\n' '\n' <English-Mix/Bible.test | sort -u |
 wc -l
2041
ahmad@DESKTOP-5SVTQJS:/mnt/c/Users/Ahmad Zuhd/Documents/4 year/NLP/ENCS5342-Assignment02-T1252/ENCS5342-Assignment02-T1252$ wc -w < English-Mix/Bible.test.tok
13999
ahmad@DESKTOP-5SVTQJS:/mnt/c/Users/Ahmad Zuhd/Documents/4 year/NLP/ENCS5342-Assignment02-T1252/ENCS5342-Assignment02-T1252$ tr ' \t\n' '\n' <English-Mix/Bible.test.tok | sort
-u | wc -l
1367
ahmad@DESKTOP-5SVTQJS:/mnt/c/Users/Ahmad Zuhd/Documents/4 year/NLP/ENCS5342-Assignment02-T1252/ENCS5342-Assignment02-T1252$
```

File	Tokens (total)	Types (unique)
UNCorpus.train (raw)	2,600,422	36,466
UNCorpus.train.tok	2,933,681	17,860
UNCorpus.test (raw)	16,693	2,695
UNCorpus.test.tok	19,169	2,155
Bible.test (raw)	11,995	2,041
Bible.test.tok	13,999	1,367

Observations:

- Tokens INCREASE after tokenization (+13% for train corpus): each punctuation mark that was attached to a word becomes a separate token, so the total word count rises.
- Types DECREASE dramatically after tokenization (−51% for train corpus): before tokenization, "word", "word,", "word." and "word;" were counted as four different types. After tokenization they all become the single type "word" (plus separate punctuation types), greatly reducing vocabulary size.
- A smaller vocabulary (fewer types) means the language model will have better coverage and fewer OOV words at test time.

Step 2 — Build the Language Model

```
##################################################
Second - Create the LM
ngram-count -text English-Mix/UNCorpus.train.tok -order 3 -lm English-Mix/UNCorpus.train.tok.3.lm -addsmooth 0.01
##################################################
```

```
$ task1.sh        UNCorpus.train.tok.3.lm ×
English-Mix > UNCorpus.train.tok.3.lm
1
2   \data\
3   ngram 1=17862
4   ngram 2=174767
5   ngram 3=255058
```

Q1.c — N-gram Entry Counts in the LM File

1-grams	2-grams	3-grams
17,862	174,767	255,058

Q1.d — Purpose of Each Column in the 1-gram Section

In the 1-gram section, each line consists of three columns. The first column represents the log10 probability of the word. The second column is the word (token) itself. The third column represents the back-off weight, which is used when the model backs off to lower-order n-grams when higher-order probabilities are not available.

Column	Name	Meaning
1st	$\log_{10}(P(w))$	Log-probability of the word
2nd	word	The word/token itself
3rd	$\log_{10}(bow(w))$	Back-off weight: used when backing off to a lower-order n-gram

Example from the LM file:

-2.653553 " -1.227169

Meaning: $\log_{10}(P('')) = -2.653553$, back-off weight =-1.227169

Q1.e — Why Does the 3-gram Section Have Only Two Columns?

The 3-gram section has only two columns (log-probability + the trigram words) because the trigram is the HIGHEST order in this model. The back-off weight column is only needed for n-grams of order n < max_order, because it specifies what weight to apply when the model cannot find a higher-order match and must back off to a lower-order n-gram. Since no order above 3 exists in this model, there is no back-off from a trigram — so the third column is omitted.

Step 3 — Evaluate Perplexity

```
#################################################
Third - Compute Perplexity
ngram -lm English-Mix/UNCorpus.train.tok.3.lm -order 3 -ppl English-Mix/UNCorpus.test.tok
file English-Mix/UNCorpus.test.tok: 500 sentences, 19169 words, 16 OOVs
0 zeroprobs, logprob= -25308.24 ppl= 19.39789 ppl1= 20.95907
ngram -lm English-Mix/UNCorpus.train.tok.3.lm -order 3 -ppl English-Mix/Bible.test.tok
file English-Mix/Bible.test.tok: 500 sentences, 13999 words, 3053 OOVs
0 zeroprobs, logprob= -36285.73 ppl= 1479.676 ppl1= 2065.266
#################################################
```

Q1.f — Why Is Perplexity Higher on the Bible Test Set?

The perplexity is much lower on the UN test set (19.39) compared to the Bible test set (1479.67) because the language model was trained on the UN Corpus. The UN test set is from the same domain (in-domain), so the model can predict the words more accurately.
In contrast, the Bible test set is from a different domain (out-of-domain), with different vocabulary and writing style, which makes it harder for the model to assign accurate probabilities, resulting in a much higher perplexity.

Q1.g — OOV Rate

Formula: OOV rate = 100 × OOVs ÷ Words

Test Set	Words	OOVs	OOV Rate
UNCorpus.test.tok	19,169	16	0.083%
Bible.test.tok	13,999	3,053	21.8%

Calculation:

UN Corpus: 100 × 16 ÷ 19,169 = 0.083%

Bible: 100 × 3,053 ÷ 13,999 = 21.8%

Q1.h — Why Is the OOV Rate Different?

The training corpus is UN Corpus text. Almost all words used in the UN test set were also seen during training (only 16 OOV tokens), because both come from the same document collection and writing style.

The Bible uses completely different vocabulary: ancient Hebrew and Greek names ("Bethlehem", "Pharaoh"), archaic English words ("thou", "hath", "beget"), and religious terminology that is absent from UN documents. These words were never seen during training, so they are all marked as OOV, yielding an OOV rate of 21.8%.

Step 4 — Generate Text

```
##############################################
Fourth - For fun, generate some random UN resolution texts!
ngram -lm English-Mix/UNCorpus.train.tok.3.lm -gen 1
3 . Encourages Governments , experts require multitude Commission responsible eradication and requests the implementation of the progress achieved in the Field the resolve / 2
90 by , inter Israeli settlers the support provided deep at the 69th ;
```

Q1.i — Comment on the Generated Text

The generated text from my run is:

“Encourages Governments, experts require multitude Commission responsible eradication and requests the implementation of the progress achieved in the Field the resolve by inter Israeli settlers the support provided deep at the 69th.”

The sentence shows some level of local fluency, as many word sequences (such as “Encourages Governments” and “implementation of the progress”) are common in UN-style language and appear natural when considered locally.

However, the global coherence is weak, since the overall sentence does not convey a clear or meaningful idea. The sentence structure is inconsistent and contains unrelated phrases joined together. This is expected because a 3-gram language model only considers a limited context (two previous words), so it cannot maintain long-range structure or meaning across the entire sentence.

Task 2 — How Many Ways to Build a Language Model?

➔ Run Variables

```
============================================================
PREPROCESSING — Create all tokenization variants
============================================================
[1/4] Tokenizing...
Tokenizer Version 1.1
Language: en
Number of threads: 1
Tokenizer Version 1.1
Language: en
Number of threads: 1
Tokenizer Version 1.1
Language: en
Number of threads: 1
Tokenizer Version 1.1
Language: en
Number of threads: 1
[2/4] Lowercasing...
[3/4] Stemming (Porter)...
[4/4] BPE...
100%|##########| 10000/10000 [00:04<00:00, 2458.56it/s]
Preprocessing complete.
```

Experiment 2.1 — Effect of Tokenization

```
============================================================
--- Training: English-Mix/UNCorpus.train ---
file English-Mix/UNCorpus.test: 500 sentences, 16693 words, 119 OOVs
0 zeroprobs, logprob= -16766.69 ppl= 9.594028 ppl1= 10.27131
file English-Mix/Bible.test: 500 sentences, 11995 words, 3607 OOVs
0 zeroprobs, logprob= -29337.12 ppl= 1998.74 ppl1= 3144.206
file English-Mix/Fair.test: 499 sentences, 4388 words, 1849 OOVs
0 zeroprobs, logprob= -10478.66 ppl= 2813.184 ppl1= 13399.33

--- Training: English-Mix/UNCorpus.train.tok ---
file English-Mix/UNCorpus.test.tok: 500 sentences, 19169 words, 16 OOVs
0 zeroprobs, logprob= -18394.8 ppl= 8.629378 ppl1= 9.128801
file English-Mix/Bible.test.tok: 500 sentences, 13999 words, 3053 OOVs
0 zeroprobs, logprob= -35900.79 ppl= 1369.418 ppl1= 1904.624
file English-Mix/Fair.test.tok: 499 sentences, 5648 words, 1761 OOVs
0 zeroprobs, logprob= -14608.77 ppl= 2141.772 ppl1= 5732.798

--- Training: English-Mix/UNCorpus.train.tok.lc ---
file English-Mix/UNCorpus.test.tok.lc: 500 sentences, 19169 words, 11 OOVs
0 zeroprobs, logprob= -18626.86 ppl= 8.86229 ppl1= 9.381574
file English-Mix/Bible.test.tok.lc: 500 sentences, 13999 words, 2797 OOVs
0 zeroprobs, logprob= -35438.28 ppl= 1067.567 ppl1= 1457.362
file English-Mix/Fair.test.tok.lc: 499 sentences, 5648 words, 1559 OOVs
0 zeroprobs, logprob= -15690.82 ppl= 2630.08 ppl1= 6875.812

--- Training: English-Mix/UNCorpus.train.tok.lc.port ---
file English-Mix/UNCorpus.test.tok.lc.port: 500 sentences, 19169 words, 7 OOVs
0 zeroprobs, logprob= -18892.81 ppl= 9.13859 ppl1= 9.681702
file English-Mix/Bible.test.tok.lc.port: 500 sentences, 13999 words, 2480 OOVs
0 zeroprobs, logprob= -36715.51 ppl= 1134.46 ppl1= 1539.526
file English-Mix/Fair.test.tok.lc.port: 499 sentences, 5648 words, 1465 OOVs
0 zeroprobs, logprob= -15905.53 ppl= 2495.547 ppl1= 6344.855

--- Training: English-Mix/UNCorpus.train.tok.lc.bpe ---
file English-Mix/UNCorpus.test.tok.lc.bpe: 500 sentences, 19447 words, 0 OOVs
0 zeroprobs, logprob= -18877.35 ppl= 8.83844 ppl1= 9.347765
file English-Mix/Bible.test.tok.lc.bpe: 500 sentences, 19192 words, 159 OOVs
0 zeroprobs, logprob= -69606.11 ppl= 3660.273 ppl1= 4540.75
file English-Mix/Fair.test.tok.lc.bpe: 499 sentences, 8123 words, 257 OOVs
```

Q2.1.a — Results Table

1) **UNCorpus.test**

Test File	Words	OOV	OOV %	PPL
UNCorpus.test (raw)	16693	119	0.713%	9.59
UNCorpus.test.tok	19169	16	0.083%	8.63
UNCorpus.test.tok.lc	19169	11	0.057%	8.86
UNCorpus.test.tok.lc.port	19169	7	0.036%	9.14
UNCorpus.test.tok.lc.bpe	19447	0	0.000%	8.84

2) **Bible.test**

Test File	Words	OOV	OOV %	PPL
Bible.test (raw)	11995	3607	30.07%	1998.74
Bible.test.tok	13999	3053	21.81%	1369.42
Bible.test.tok.lc	13999	2797	19.98%	1067.57
Bible.test.tok.lc.port	13999	2480	17.72%	1134.46
Bible.test.tok.lc.bpe	19192	159	0.83%	3660.27

3) Fair.test

Test File	Words	OOV	OOV %	PPL
Fair.test (raw)	4388	1849	42.14%	2813.18
Fair.test.tok	5648	1761	31.18%	2141.77
Fair.test.tok.lc	5648	1559	27.60%	2630.08
Fair.test.tok.lc.port	5648	1465	25.94%	2495.55
Fair.test.tok.lc.bpe	8123	257	3.16%	8216.88

Q2.1.b — Tokenization vs OOV Rate

In-domain (UNCorpus.test):
The OOV rate starts at **0.71% (raw)** and decreases consistently with preprocessing, reaching **0% with BPE**. Lowercasing reduces case-based variations, stemming merges morphological variants, and BPE eliminates OOV entirely by decomposing words into known subword units. The improvement is smooth because the test data comes from the same domain as the training data.

Out-of-domain (Bible, Fair):
The OOV rate is much higher initially (**30.07% for Bible, 42.14% for Fair**) due to domain mismatch. It decreases gradually with tokenization, lowercasing, and stemming (down to **17.72% for Bible** and **25.94% for Fair**). With BPE, the OOV rate drops dramatically (**0.83% for Bible, 3.16% for Fair**) because most unseen words can be segmented into known subword units. However, unlike the in-domain case, OOV does not fully disappear because some subword patterns are still unseen.

Q2.1.c — Tokenization vs Perplexity

In-domain (UNCorpus.test):
Perplexity decreases slightly from **9.59 (raw)** to around **8.6–8.8 (tokenized/BPE)**. The improvement is moderate because the model already performs well on in-domain data. Lower OOV and normalized vocabulary allow the model to assign slightly better probabilities.

Out-of-domain (Bible, Fair):
Perplexity drops significantly from raw to tokenized and lowercased forms:

- Bible: **1998 → 1067 (tok.lc)**
- Fair: **2813 → ~2495 (porter)**

This shows that preprocessing helps reduce mismatch.

However, with **BPE**, perplexity increases sharply:

- Bible: **3660**
- Fair: very high (≈ **>1600–3000 range**)

This happens because BPE changes the sequence structure by splitting words into subword units. Although OOV is reduced, the model now sees longer sequences and different n-gram patterns that do not match the training distribution well.

Q2.1.d — Bible vs Fair Lady

Across all preprocessing steps:

- **Fair.test consistently has higher OOV rates** than Bible
  (e.g., raw: **42.14% vs 30.07%**, porter: **25.94% vs 17.72%**)

- **Fair.test also has higher perplexity** than Bible in most cases
  (e.g., raw: **2813 vs 1998**, porter: **2495 vs 1134**)

This suggests that My Fair Lady is more distant from the UN Corpus than the Bible is, both in vocabulary and writing style, since it contains colloquial British dialogue that is completely absent from formal diplomatic text.

Experiment 2.2 — Effect of Training Size

```
--- Training size: 70000 lines ---
file English-Mix/UNCorpus.test.tok.lc: 500 sentences, 19169 words, 11 OOVs
0 zeroprobs, logprob= -18626.86 ppl= 8.86229 ppl1= 9.381574
file English-Mix/Bible.test.tok.lc: 500 sentences, 13999 words, 2797 OOVs
0 zeroprobs, logprob= -35438.28 ppl= 1067.567 ppl1= 1457.362
file English-Mix/Fair.test.tok.lc: 499 sentences, 5648 words, 1559 OOVs
0 zeroprobs, logprob= -15690.82 ppl= 2630.08 ppl1= 6875.812

--- Training size: 35000 lines ---
file English-Mix/UNCorpus.test.tok.lc: 500 sentences, 19169 words, 87 OOVs
0 zeroprobs, logprob= -24370.19 ppl= 17.55982 ppl1= 18.9291
file English-Mix/Bible.test.tok.lc: 500 sentences, 13999 words, 3097 OOVs
0 zeroprobs, logprob= -33065.75 ppl= 794.3207 ppl1= 1078.944
file English-Mix/Fair.test.tok.lc: 499 sentences, 5648 words, 1649 OOVs
0 zeroprobs, logprob= -14771.11 ppl= 1922.777 ppl1= 4939.712

--- Training size: 17500 lines ---
file English-Mix/UNCorpus.test.tok.lc: 500 sentences, 19169 words, 190 OOVs
0 zeroprobs, logprob= -25911.18 ppl= 21.39002 ppl1= 23.18758
file English-Mix/Bible.test.tok.lc: 500 sentences, 13999 words, 3360 OOVs
0 zeroprobs, logprob= -30915.72 ppl= 596.2771 ppl1= 805.1663
file English-Mix/Fair.test.tok.lc: 499 sentences, 5648 words, 1812 OOVs
0 zeroprobs, logprob= -13595.57 ppl= 1368.466 ppl1= 3501.111

--- Training size: 8750 lines ---
file English-Mix/UNCorpus.test.tok.lc: 500 sentences, 19169 words, 259 OOVs
0 zeroprobs, logprob= -28734.62 ppl= 30.22754 ppl1= 33.07853
file English-Mix/Bible.test.tok.lc: 500 sentences, 13999 words, 3543 OOVs
0 zeroprobs, logprob= -29122.56 ppl= 455.1322 ppl1= 609.8854
file English-Mix/Fair.test.tok.lc: 499 sentences, 5648 words, 1909 OOVs
0 zeroprobs, logprob= -12685.62 ppl= 984.7 ppl1= 2470.499

--- Training size: 4375 lines ---
file English-Mix/UNCorpus.test.tok.lc: 500 sentences, 19169 words, 793 OOVs
0 zeroprobs, logprob= -36347.51 ppl= 84.25468 ppl1= 95.05791
file English-Mix/Bible.test.tok.lc: 500 sentences, 13999 words, 3727 OOVs
0 zeroprobs, logprob= -28360.84 ppl= 429.3675 ppl1= 576.7463
file English-Mix/Fair.test.tok.lc: 499 sentences, 5648 words, 2087 OOVs
0 zeroprobs, logprob= -11807.44 ppl= 809.5354 ppl1= 2069.019
```

Q2.2.a — Results Table

Test File	Train Lines	Words	OOV	OOV %	PPL
UNCorpus.test.tok.lc	70,000	19169	11	0.057%	8.86
UNCorpus.test.tok.lc	35,000	19169	87	0.454%	17.56
UNCorpus.test.tok.lc	17,500	19169	190	0.991%	21.39
UNCorpus.test.tok.lc	8,750	19169	259	1.351%	30.23
UNCorpus.test.tok.lc	4,375	19169	793	4.136%	84.25

Bible.test.tok.lc	70,000	13999	2797	19.98%	1067.57
Bible.test.tok.lc	35,000	13999	3097	22.12%	794.32
Bible.test.tok.lc	17,500	13999	3360	24.00%	596.28
Bible.test.tok.lc	8,750	13999	3543	25.31%	455.13
Bible.test.tok.lc	4,375	13999	3727	26.63%	429.37
Fair.test.tok.lc	70,000	5648	1559	27.60%	2630.08
Fair.test.tok.lc	35,000	5648	1649	29.20%	1922.78
Fair.test.tok.lc	17,500	5648	1812	32.08%	1368.47
Fair.test.tok.lc	8,750	5648	1909	33.80%	984.70
Fair.test.tok.lc	4,375	5648	2087	36.96%	809.54

Q2.2.b — Training Size vs OOV Rate

As the training size decreases, the OOV rate increases consistently for all three test sets. This is expected because a smaller training corpus contains fewer unique words, so more test words are unseen.

For the **in-domain data (UNCorpus.test.tok.lc)**, the OOV rate increases sharply from **0.057% (70k lines)** to **4.136% (4,375 lines)**. This happens because the in-domain vocabulary is well covered when using the full dataset, but quickly becomes incomplete as training data is reduced.

For the **out-of-domain datasets (Bible and Fair)**, the OOV rate is already high even at full training size (**~20–28%**) and increases more gradually as the training size decreases (up to **~26–37%**). This is because many words are inherently different from the UN corpus and remain unseen regardless of training size.

Q2.2.c — Training Size vs Perplexity (In-domain)

For the **in-domain test set**, perplexity increases significantly as training size decreases. At **70k lines**, the perplexity is **≈ 8.86**, while at **4,375 lines**, it rises to **≈ 84.25** — almost a **10× increase**.

This behavior is expected because a larger training set provides better coverage of n-gram patterns and more accurate probability estimates. With less data, the model becomes less confident and assigns lower probabilities to test sequences, resulting in higher perplexity. The relationship is smooth and monotonic.

Q2.2.d — Training Size vs Perplexity (Out-of-domain)

For the **out-of-domain datasets (Bible and Fair)**, perplexity decreases as training size decreases:

- Bible: **1067 → 429**

- Fair: **2630 → 809**

This counter-intuitive behavior occurs because with less training data, the language model becomes simpler and relies more on lower-order n-grams (e.g., unigrams and bigrams). These lower-order models are more general and less domain-specific, so they assign more uniform probabilities to unseen or mismatched text.

In contrast, a model trained on a large dataset learns more specific higher-order patterns that are strongly tied to the UN domain. When evaluated on out-of-domain text, these specific patterns do not match well, leading to lower probabilities and higher perplexity.

Experiment 2.3 — Effect of LM Order

```
============================================================
--- Order: 1 ---
file English-Mix/UNCorpus.test.tok.lc: 500 sentences, 19169 words, 11 OOVs
0 zeroprobs, logprob= -51469.71 ppl= 415.2004 ppl1= 485.9493
file English-Mix/Bible.test.tok.lc: 500 sentences, 13999 words, 2797 OOVs
0 zeroprobs, logprob= -32725.57 ppl= 626.0066 ppl1= 834.4568
file English-Mix/Fair.test.tok.lc: 499 sentences, 5648 words, 1559 OOVs
0 zeroprobs, logprob= -14289.84 ppl= 1302 ppl1= 3123.934

--- Order: 2 ---
file English-Mix/UNCorpus.test.tok.lc: 500 sentences, 19169 words, 11 OOVs
0 zeroprobs, logprob= -29776.39 ppl= 32.71305 ppl1= 35.83054
file English-Mix/Bible.test.tok.lc: 500 sentences, 13999 words, 2797 OOVs
0 zeroprobs, logprob= -34619.67 ppl= 908.7421 ppl1= 1231.659
file English-Mix/Fair.test.tok.lc: 499 sentences, 5648 words, 1559 OOVs
0 zeroprobs, logprob= -15479.82 ppl= 2365.807 ppl1= 6105.511

--- Order: 3 ---
file English-Mix/UNCorpus.test.tok.lc: 500 sentences, 19169 words, 11 OOVs
0 zeroprobs, logprob= -18626.86 ppl= 8.86229 ppl1= 9.381574
file English-Mix/Bible.test.tok.lc: 500 sentences, 13999 words, 2797 OOVs
0 zeroprobs, logprob= -35438.28 ppl= 1067.567 ppl1= 1457.362
file English-Mix/Fair.test.tok.lc: 499 sentences, 5648 words, 1559 OOVs
0 zeroprobs, logprob= -15690.82 ppl= 2630.08 ppl1= 6875.812

--- Order: 4 ---
file English-Mix/UNCorpus.test.tok.lc: 500 sentences, 19169 words, 11 OOVs
0 zeroprobs, logprob= -13585.14 ppl= 4.909919 ppl1= 5.11812
file English-Mix/Bible.test.tok.lc: 500 sentences, 13999 words, 2797 OOVs
0 zeroprobs, logprob= -35547.57 ppl= 1090.773 ppl1= 1490.47
file English-Mix/Fair.test.tok.lc: 499 sentences, 5648 words, 1559 OOVs
0 zeroprobs, logprob= -15703.14 ppl= 2646.398 ppl1= 6923.695

--- Order: 5 ---
file English-Mix/UNCorpus.test.tok.lc: 500 sentences, 19169 words, 11 OOVs
0 zeroprobs, logprob= -11564.54 ppl= 3.875133 ppl1= 4.014581
file English-Mix/Bible.test.tok.lc: 500 sentences, 13999 words, 2797 OOVs
0 zeroprobs, logprob= -35556.8 ppl= 1092.756 ppl1= 1493.3
file English-Mix/Fair.test.tok.lc: 499 sentences, 5648 words, 1559 OOVs
0 zeroprobs, logprob= -15704.34 ppl= 2647.987 ppl1= 6928.359
```

Q2.3.a — Results Table

Test File	Order	Words	OOV	OOV %	PPL
UNCorpus.test.tok.lc	1	19169	11	0.057%	415.20
UNCorpus.test.tok.lc	2	19169	11	0.057%	32.71
UNCorpus.test.tok.lc	3	19169	11	0.057%	8.86
UNCorpus.test.tok.lc	4	19169	11	0.057%	4.91
UNCorpus.test.tok.lc	5	19169	11	0.057%	3.88

Bible.test.tok.lc	1	13999	2797	19.98%	626.01
Bible.test.tok.lc	2	13999	2797	19.98%	908.74
Bible.test.tok.lc	3	13999	2797	19.98%	1067.57
Bible.test.tok.lc	4	13999	2797	19.98%	1090.77
Bible.test.tok.lc	5	13999	2797	19.98%	1092.76
Fair.test.tok.lc	1	5648	1559	27.60%	1302.00
Fair.test.tok.lc	2	5648	1559	27.60%	2365.81
Fair.test.tok.lc	3	5648	1559	27.60%	2630.08
Fair.test.tok.lc	4	5648	1559	27.60%	2646.40
Fair.test.tok.lc	5	5648	1559	27.60%	2647.99

Q2.3.b — LM Order vs OOV Rate

The OOV rate is independent of the language model order. It depends only on the training vocabulary, not on how probabilities are computed.

From the results, the OOV counts remain constant across all orders:

- UNCorpus: **11 OOVs**
- Bible: **2797 OOVs**
- Fair: **1559 OOVs**

This is because changing the n-gram order (1 to 5) does not change which words were seen during training — it only changes how probabilities are estimated.

Q2.3.c — Order vs Perplexity (In-domain)

For the in-domain dataset (UNCorpus.test.tok.lc), perplexity decreases dramatically as the order increases:

- Order 1: **415.20**
- Order 2: **32.71**
- Order 3: **8.86**
- Order 4: **4.91**
- Order 5: **3.88**

This shows a strong monotonic improvement. Higher-order models capture more context (longer word sequences), allowing the model to assign more accurate probabilities to in-domain text.

Higher-order models (4, 5) keep improving on in-domain data because Witten-Bell smoothing handles data sparsity well, so even sparse 4-gram and 5-gram counts are useful rather than harmful. This indicates that the available training data is sufficient to support higher-order n-grams in the in-domain case.

Q2.3.d — Order vs Perplexity (Out-of-domain)

For out-of-domain datasets (Bible and Fair), perplexity behaves differently:

Bible:

- Order 1: **626.01**
- Order 2: **908.74**
- Order 3: **1067.57**
- Order 4: **1090.77**
- Order 5: **1092.76**

Fair:

- Order 1: **1302.00**
- Order 2: **2365.81**
- Order 3: **2630.08**
- Order 4: **2646.40**
- Order 5: **2647.99**

Here, perplexity **increases as order increases**.

This happens because higher-order n-grams capture very specific patterns from the UN domain. These patterns do not match the out-of-domain texts (Bible and Fair), so the model frequently backs off and assigns lower probabilities, resulting in higher perplexity.

Lower-order models (especially unigram) are more general and assign smoother probabilities, which accidentally work better for out-of-domain data.

Experiment 2.4 — Effect of Smoothing Method

```
--- Add-1 (Laplace) ---
file English-Mix/UNCorpus.test.tok.lc: 500 sentences, 19169 words, 11 OOVs
0 zeroprobs, logprob= -57687.42 ppl= 860.106 ppl1= 1025.983
file English-Mix/Bible.test.tok.lc: 500 sentences, 13999 words, 2797 OOVs
0 zeroprobs, logprob= -35607.02 ppl= 1103.608 ppl1= 1508.797
file English-Mix/Fair.test.tok.lc: 499 sentences, 5648 words, 1559 OOVs
0 zeroprobs, logprob= -14786.44 ppl= 1670.519 ppl1= 4131.914
--- Add-0.1 ---
file English-Mix/UNCorpus.test.tok.lc: 500 sentences, 19169 words, 11 OOVs
0 zeroprobs, logprob= -41154.38 ppl= 124.0276 ppl1= 140.6554
file English-Mix/Bible.test.tok.lc: 500 sentences, 13999 words, 2797 OOVs
0 zeroprobs, logprob= -34987.08 ppl= 976.8714 ppl1= 1328.276
file English-Mix/Fair.test.tok.lc: 499 sentences, 5648 words, 1559 OOVs
0 zeroprobs, logprob= -14956.75 ppl= 1819.579 ppl1= 4547.791
--- Good-Turing (SRILM default) ---
warning: discount coeff 1 is out of range: 0
warning: discount coeff 6 is out of range: 1.19324
warning: discount coeff 7 is out of range: 1.14295
warning: discount coeff 6 is out of range: 1.20002
warning: discount coeff 7 is out of range: 1.08969
warning: discount coeff 6 is out of range: 1.21451
warning: discount coeff 7 is out of range: 1.06186
warning: discount coeff 6 is out of range: 1.24362
warning: discount coeff 7 is out of range: 1.04265
file English-Mix/UNCorpus.test.tok.lc: 500 sentences, 19169 words, 11 OOVs
0 zeroprobs, logprob= -11757.44 ppl= 3.96369 ppl1= 4.108747
file English-Mix/Bible.test.tok.lc: 500 sentences, 13999 words, 2797 OOVs
0 zeroprobs, logprob= -37136.96 ppl= 1491.27 ppl1= 2066.368
file English-Mix/Fair.test.tok.lc: 499 sentences, 5648 words, 1559 OOVs
0 zeroprobs, logprob= -16408.66 ppl= 3770.759 ppl1= 10300.95
--- Witten-Bell ---
file English-Mix/UNCorpus.test.tok.lc: 500 sentences, 19169 words, 11 OOVs
0 zeroprobs, logprob= -11564.54 ppl= 3.875133 ppl1= 4.014581
file English-Mix/Bible.test.tok.lc: 500 sentences, 13999 words, 2797 OOVs
0 zeroprobs, logprob= -35556.8 ppl= 1092.756 ppl1= 1493.3
file English-Mix/Fair.test.tok.lc: 499 sentences, 5648 words, 1559 OOVs
0 zeroprobs, logprob= -15704.34 ppl= 2647.987 ppl1= 6928.359
--- Kneser-Ney ---
```

```
--- Kneser-Ney ---
file English-Mix/UNCorpus.test.tok.lc: 500 sentences, 19169 words, 11 OOVs
0 zeroprobs, logprob= -11882.29 ppl= 4.022078 ppl1= 4.170863
file English-Mix/Bible.test.tok.lc: 500 sentences, 13999 words, 2797 OOVs
0 zeroprobs, logprob= -32173.63 ppl= 561.582 ppl1= 744.96
file English-Mix/Fair.test.tok.lc: 499 sentences, 5648 words, 1559 OOVs
0 zeroprobs, logprob= -14109.06 ppl= 1189.075 ppl1= 2821.575

============================================================
```

Q2.4.a — Results Table

Test File	Smoothing	Words	OOV	OOV %	PPL
UNCorpus.test.tok.lc	Add-1 (Laplace)	19169	11	0.057%	860.11
UNCorpus.test.tok.lc	Add-0.1	19169	11	0.057%	124.03
UNCorpus.test.tok.lc	Good-Turing	19169	11	0.057%	3.96
UNCorpus.test.tok.lc	Witten-Bell	19169	11	0.057%	3.88
UNCorpus.test.tok.lc	Kneser-Ney	19169	11	0.057%	4.02
Bible.test.tok.lc	Add-1 (Laplace)	13999	2797	19.98%	1103.61
Bible.test.tok.lc	Add-0.1	13999	2797	19.98%	976.87
Bible.test.tok.lc	Good-Turing	13999	2797	19.98%	1491.27
Bible.test.tok.lc	Witten-Bell	13999	2797	19.98%	1092.76
Bible.test.tok.lc	Kneser-Ney	13999	2797	19.98%	561.58
Fair.test.tok.lc	Add-1 (Laplace)	5648	1559	27.60%	1670.52
Fair.test.tok.lc	Add-0.1	5648	1559	27.60%	1819.58
Fair.test.tok.lc	Good-Turing	5648	1559	27.60%	3770.76
Fair.test.tok.lc	Witten-Bell	5648	1559	27.60%	2647.99
Fair.test.tok.lc	Kneser-Ney	5648	1559	27.60%	1189.08

Q2.4.b — Smoothing vs Perplexity (In-domain)

For the in-domain dataset (UNCorpus.test.tok.lc), perplexity improves significantly as we move from simple to more advanced smoothing methods:

- Add-1: **860.11** (worst)

- Add-0.1: **124.03**
- Good-Turing: **3.96**
- Witten-Bell: **3.88 (best)**
- Kneser-Ney: **4.02**

Add-1 performs poorly because it over-smooths by assigning too much probability mass to unseen events. Add-0.1 reduces this effect but still performs much worse than the other methods.

The advanced methods (Good-Turing, Witten-Bell, and Kneser-Ney) perform significantly better. In your results, **Witten-Bell achieves the lowest perplexity**, slightly outperforming Kneser-Ney. This indicates that for this dataset size and domain, Witten-Bell provides the best balance between smoothing and preserving useful probability structure.

Q2.4.c — Smoothing vs Perplexity (Out-of-domain)

For out-of-domain datasets (Bible and Fair), the behavior differs across methods:

Bible:

- Kneser-Ney: **561.58 (best)**
- Add-0.1: **976.87**
- Witten-Bell: **1092.76**
- Add-1: **1103.61**
- Good-Turing: **1491.27 (worst)**

Fair:

- Kneser-Ney: **1189.08 (best)**
- Add-1: **1670.52**
- Add-0.1: **1819.58**
- Witten-Bell: **2647.99**
- Good-Turing: **3770.76 (worst)**

Kneser-Ney clearly performs best on both out-of-domain datasets. This is because it uses **continuation probabilities**, which better capture how words appear across different contexts, improving generalization to unseen domains.

Good-Turing performs worst, especially on Fair, likely due to instability in discount estimates (as indicated by warnings during execution).

Q2.4.d — Best Smoothing Method Overall

The best smoothing method depends on the evaluation domain:

- **In-domain (UNCorpus):**
  → **Witten-Bell performs best** (lowest perplexity)
- **Out-of-domain (Bible, Fair):**
  → **Kneser-Ney performs best** (lowest perplexity)

Overall, **Kneser-Ney is the best general-purpose method**, especially when dealing with unknown or out-of-domain data. This is because it estimates probabilities based on **continuation counts**, which capture how widely a word appears across contexts, improving robustness.

However, Witten-Bell remains a strong alternative and performs slightly better in the in-domain setting.

Task 3 — Guess the Language!

Q3.1 — Language Identification System

System Design

The system uses character-level trigram language models (LM order = 3) with Witten-Bell smoothing, one model per language. Every character in a text line — including the space character (represented as <SP>) — is treated as a separate token. This character-level approach works well for language identification because:

- Each language has a distinctive character distribution and n-gram frequency profile
- Character models handle morphological variation naturally, without needing word segmentation
- Character-level models generalise across scripts — languages with unique alphabets (Greek, Bulgarian) are immediately distinguishable from Latin-script languages
- Trigram context (3 characters) captures typical letter combinations unique to each language

How the Script Works

TRAIN mode:

- Reads each training file (train.<lang>)
- Converts every line to a space-separated character sequence where spaces become <SP>
- Runs ngram-count with -order 3 -wbdiscount to build the LM
- Saves the .lm file to the model directory

PREDICT mode:

- For each test file with a numeric extension (dev.1, test.3, etc.)
- Character-tokenizes the file the same way as training
- Runs ngram -ppl against all 22 language LMs
- Outputs the language code with the lowest perplexity

EVALUATE mode:

- Compares predictions against the gold file and reports accuracy with a list of any errors

Development Set Results

```
============================================================
QUESTION 3.1 — Full model: character trigram + Witten-Bell
============================================================

[Step 1] Training language models (all 22 languages)...
[TRAIN] Building LM for language: it
[TRAIN] Building LM for language: lt
[TRAIN] Building LM for language: et
[TRAIN] Building LM for language: fr
[TRAIN] Building LM for language: hu
[TRAIN] Building LM for language: lv
[TRAIN] Building LM for language: cs
[TRAIN] Building LM for language: en
[TRAIN] Building LM for language: da
[TRAIN] Building LM for language: de
[TRAIN] Building LM for language: mt
[TRAIN] Building LM for language: nl
[TRAIN] Building LM for language: pl
[TRAIN] Building LM for language: fi
[TRAIN] Building LM for language: pt
[TRAIN] Building LM for language: ro
[TRAIN] Building LM for language: sk
[TRAIN] Building LM for language: sl
[TRAIN] Building LM for language: sv
[TRAIN] Building LM for language: bg
[TRAIN] Building LM for language: el
[TRAIN] Building LM for language: es
[TRAIN] Done. Models saved to: modeldir

[Step 2] Predicting development set...

[Step 3] Evaluating on development set...
Accuracy: 22/22 = 100.00%
```

Question 3.2 — Reduced Training and Evaluation

```
============================================================

[Step 1] Training with 1 random line per language (order 1)...
[TRAIN] Building LM for language: it
[TRAIN] Building LM for language: lt
[TRAIN] Building LM for language: et
[TRAIN] Building LM for language: fr
[TRAIN] Building LM for language: hu
[TRAIN] Building LM for language: lv
[TRAIN] Building LM for language: cs
[TRAIN] Building LM for language: en
[TRAIN] Building LM for language: da
[TRAIN] Building LM for language: de
[TRAIN] Building LM for language: mt
[TRAIN] Building LM for language: nl
[TRAIN] Building LM for language: pl
[TRAIN] Building LM for language: fi
[TRAIN] Building LM for language: pt
[TRAIN] Building LM for language: ro
[TRAIN] Building LM for language: sk
[TRAIN] Building LM for language: sl
[TRAIN] Building LM for language: sv
[TRAIN] Building LM for language: bg
[TRAIN] Building LM for language: el
[TRAIN] Building LM for language: es
[TRAIN] Done. Models saved to: modeldir_small

[Step 2] Predicting development set (1 random line from each dev file)...

[Step 3] Evaluating reduced model...
Accuracy: 1/22 = 4.55%
```

```
Errors:
  dev.1: true=it, predicted=lv
  dev.10: true=de, predicted=lv
  dev.11: true=mt, predicted=fi
  dev.12: true=nl, predicted=fi
  dev.13: true=pl, predicted=fi
  dev.15: true=pt, predicted=fi
  dev.16: true=ro, predicted=fi
  dev.17: true=sk, predicted=lv
  dev.18: true=sl, predicted=lv
  dev.19: true=sv, predicted=lv
  dev.2: true=lt, predicted=fi
  dev.20: true=bg, predicted=lv
  dev.21: true=el, predicted=cs
  dev.22: true=es, predicted=fi
  dev.3: true=et, predicted=fi
  dev.4: true=fr, predicted=fi
  dev.5: true=hu, predicted=lv
  dev.6: true=lv, predicted=fi
  dev.7: true=cs, predicted=lv
  dev.8: true=en, predicted=bg
  dev.9: true=da, predicted=fi
```

Analysis of Error Pattern

The accuracy drops dramatically from **100% to 4.55% (1/22)**. The error pattern is highly systematic: most misclassified files are predicted as **Finnish (fi)** and **Latvian (lv)**, with a few predictions as other languages such as Czech (cs) and Bulgarian (bg).

This happens for several reasons:

• With only **1 random training line**, the unigram language model (order = 1) contains very limited character coverage. Many characters that appear in the test files are not seen during training.

• Languages such as **Finnish (fi) and Latvian (lv)** happen to include relatively common Latin characters in the sampled training lines, allowing them to assign non-zero probabilities to a wider range of test characters. As a result, they become the model's "default" predictions.

• Using **order = 1 removes all sequential context**. The model only considers individual character frequencies and ignores important character sequences (such as "th", "sch", "qu"), which are essential for distinguishing between languages.

• For languages with **unique scripts** (such as Greek and Bulgarian), performance also fails because the single training line may not include all script characters. When unseen characters appear in the test data, they behave like OOV tokens, making the model unreliable.

• The predictions become biased toward a few languages that happen to have slightly better character overlap with many test files, rather than reflecting true linguistic similarity.

Summary:
The experiment shows that language identification requires both **sufficient training data** and **higher-order n-grams**. Without enough data and context, the model becomes unreliable and biased toward a small subset of languages.

Question 3.3 — Blind Test Set Predictions

Model Configuration Used

The best-performing configuration from Q3.1 was used:

- Character-level trigram language models (order = 3)
- Witten-Bell smoothing
- Full training data

This configuration achieved **100% accuracy (22/22)** on the development set.

```
============================================================
QUESTION 3.3 — Blind test set predictions
============================================================

[Step 1] Predicting blind test set using full model...

test.pred contents:
test.1  sk
test.2  nl
test.3  fi
test.4  da
test.5  de
test.6  mt
test.7  ro
test.8  sl
test.9  bg
test.10 en
test.11 el
test.12 it
test.13 fr
test.14 cs
test.15 pl

============================================================
Task 3 complete!
Output files: dev.predict, dev.predict.small, test.pred
============================================================
```

The test.pred file contains these 15 predictions in the required tab-separated format.

Conclusion

The model successfully generates predictions for all test files using the best configuration.
Since the gold labels for the test set are not provided, accuracy cannot be directly measured. However, based on the **100% accuracy achieved on the development set**, the model is expected to generalize well to unseen data.

This confirms that **character-level trigram models with Witten-Bell smoothing are highly effective for language identification tasks**.